

Machine Learning - HW #1

Snake Game + Tic-Tac-Toe

Student Names: Mana Peiravian – Sina Sabooki

Student Numbers: 40464250 - 40435072

Course: Machine Learning

Instructor: Dr. Sattar Hashemi

1. Introduction

In this assignment, we implemented learning agents for two games—Snake and Tic-tac-toe—using a function approximation approach. Instead of storing values for every possible state (which is infeasible due to the large state space), we approximate the value of a state using a linear model:

$$\hat{V}(x) = w^T x$$

where:

- x = feature vector representing the state
- w = weight vector learned during training

The agent improves its performance by iteratively updating weights based on experience using a temporal-difference-like update rule.

2. Core Learning Approach

Both parts of the assignment follow the same learning framework:

2.1 State Representation

Each game state is mapped to a feature vector:

$$x = [x_1, x_2, \dots, x_n]$$

2.2 Value Approximation

$$\hat{V}(s) = w^T x(s)$$

2.3 Action Selection (Greedy Policy)

At each state:

- Evaluate all possible actions
- Compute value of successor states
- Choose the action with the maximum estimated value

2.4 Training Target

- Non-terminal:

$$V_{train}(s_t) = \hat{V}(s_{t+1})$$

- Terminal:
 - Snake: -1000 for losing
 - Tic-tac-toe: +100 (win), -100 (loss), 0 (draw)

2.5 Weight Update Rule

$$w_i = w_i + \alpha \left(V_{train}(s_t) - \hat{V}(s_t) \right) x_i$$

3. Part I — Snake Game

3.1 Problem Description

The Snake game is modeled as a graph:

- Nodes = board states
- Edges = actions

Goal: maximize food consumption while avoiding collisions.

3.2 State & Environment Setup

- Board size: 20×20
- Snake length: 3
- Obstacles: 4 blocks (2×2)
- One food item
- Random initial configuration

3.3 Feature Design (Critical Part)

To avoid poor performance (like looping), we designed rich features:

Basic Features

- Bias term = 1
- Relative food position:

- food above / below / left / right
- Distance to food (normalized)

Safety Features

- Danger straight, left, right
- Immediate collision detection

Advanced Features (Improvement over baseline)

These were crucial to prevent looping behavior:

1. Free Space Estimation (Flood Fill)
 - Estimates how much space is reachable
 - Helps avoid trapping itself
2. Tail Reachability
 - Checks if snake can reach its tail
 - Prevents dead-end configurations
3. Food Reachability
 - Ensures food is accessible
4. Local Openness
 - Number of free cells in front / left / right
5. Loop Avoidance
 - Penalizes revisiting recent positions
6. Directional Progress
 - Whether move brings snake closer to food

3.4 Key Implementation Details

- Linear model with weights initialized to zero
- Greedy action selection based on successor value
- Optional ϵ -greedy exploration during training
- Terminal states assigned large negative reward

3.5 Results & Observations

Initial Behavior

- Snake moved randomly
- Frequently collided early

After Training

- Learned to:
 - Move toward food
 - Avoid immediate danger
 - Navigate around obstacles
- Reduced self-collision significantly

Episode	100/8000	avg score=2.320	best=11	epsilon=0.198
Episode	200/8000	avg score=2.350	best=12	epsilon=0.195
Episode	300/8000	avg score=2.200	best=11	epsilon=0.193
Episode	400/8000	avg score=2.280	best=10	epsilon=0.191
Episode	500/8000	avg score=2.200	best=11	epsilon=0.188
Episode	600/8000	avg score=1.950	best=9	epsilon=0.186
Episode	700/8000	avg score=2.530	best=12	epsilon=0.183
Episode	800/8000	avg score=2.830	best=11	epsilon=0.181
Episode	900/8000	avg score=2.330	best=13	epsilon=0.179
Episode	1000/8000	avg score=2.400	best=12	epsilon=0.176

...

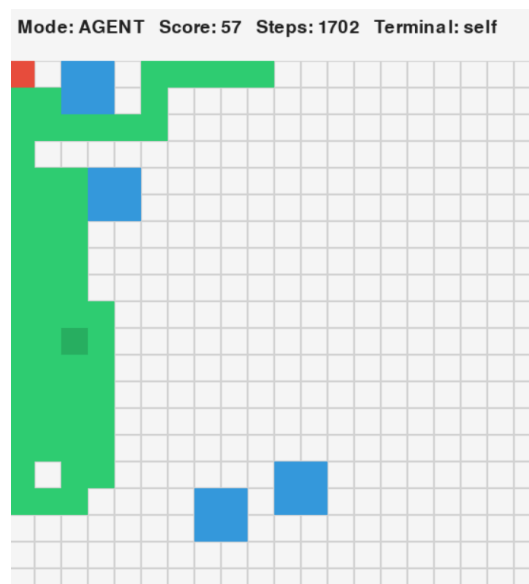
Episode 7000/8000	avg score=11.710	best=43	epsilon=0.034
Episode 7100/8000	avg score=6.770	best=32	epsilon=0.031
Episode 7200/8000	avg score=7.410	best=27	epsilon=0.029
Episode 7300/8000	avg score=8.820	best=33	epsilon=0.027
Episode 7400/8000	avg score=9.400	best=35	epsilon=0.024
Episode 7500/8000	avg score=7.410	best=34	epsilon=0.022
Episode 7600/8000	avg score=9.370	best=38	epsilon=0.020
Episode 7700/8000	avg score=10.000	best=47	epsilon=0.017
Episode 7800/8000	avg score=13.760	best=46	epsilon=0.015
Episode 7900/8000	avg score=12.220	best=37	epsilon=0.012
Episode 8000/8000	avg score=13.590	best=40	epsilon=0.010

Common Issue

- Early versions got stuck in loops
- Fixed by:
 - Adding exploration (ϵ -greedy)
 - Adding loop-penalty features
 - Using spatial awareness features

Final Performance

- Achieves moderate scores
- Still not optimal (due to linear model limitations)



4. Part II — Tic-tac-toe

4.1 Problem Description

Tic-tac-toe is a two-player zero-sum game.

Goal:

- Win (+100)
- Avoid losing (−100)
- Draw (0)

4.2 Key Difference from Snake

- Alternating turns between players
- Opponent is adversarial

Important Adjustment:

We evaluate states from the current player's perspective, and:

$$V_{train}(s_t) = -\hat{V}(s_{t+1})$$

This accounts for opponent moves.

4.3 Feature Design

Features were designed to capture strategic patterns:

Basic Features

- Bias term
- Number of own marks
- Number of opponent marks
- Empty cells

Positional Features

- Center control
- Corner control
- Edge control

Strategic Features

1. Winning Opportunities
 - Open 2-in-a-row (one move away from win)
2. Blocking Needs
 - Opponent 2-in-a-row threats
3. Fork Opportunities
 - States that create multiple winning paths
4. Line Control
 - Number of lines still winnable
5. Immediate Win Detection
 - Whether current move wins

4.4 Implementation Details

- Board encoded numerically:
 - +1 = current player
 - -1 = opponent
 - 0 = empty
- Linear model used for value estimation
- Training via self-play

4.5 Results & Observations

Initial Behavior

- Random and inconsistent
- Frequent losses

```
Feature dimension: 26
O |   | X
-----
X |   |
-----
  | X | O
Player to move: O
Estimated value: 0.0
```

After Training

- Learns to:
 - Take center early
 - Block opponent threats
 - Create winning opportunities

```
Training started...
Episode 1000/20000 | X wins: 452 | O wins: 182 | Draws: 366 | epsilon=0.1482
Episode 2000/20000 | X wins: 403 | O wins: 164 | Draws: 433 | epsilon=0.1098
Episode 3000/20000 | X wins: 521 | O wins: 149 | Draws: 330 | epsilon=0.0813
Episode 4000/20000 | X wins: 367 | O wins: 133 | Draws: 500 | epsilon=0.0602
Episode 5000/20000 | X wins: 452 | O wins: 141 | Draws: 407 | epsilon=0.0446
Episode 6000/20000 | X wins: 361 | O wins: 118 | Draws: 521 | epsilon=0.0331
Episode 7000/20000 | X wins: 457 | O wins: 108 | Draws: 435 | epsilon=0.0245
Episode 8000/20000 | X wins: 505 | O wins: 103 | Draws: 392 | epsilon=0.0181
Episode 9000/20000 | X wins: 518 | O wins: 108 | Draws: 374 | epsilon=0.0134
Episode 10000/20000 | X wins: 413 | O wins: 111 | Draws: 476 | epsilon=0.0100
Episode 11000/20000 | X wins: 460 | O wins: 106 | Draws: 434 | epsilon=0.0100
Episode 12000/20000 | X wins: 369 | O wins: 115 | Draws: 516 | epsilon=0.0100
Episode 13000/20000 | X wins: 309 | O wins: 101 | Draws: 590 | epsilon=0.0100
Episode 14000/20000 | X wins: 341 | O wins: 123 | Draws: 536 | epsilon=0.0100
Episode 15000/20000 | X wins: 316 | O wins: 95 | Draws: 589 | epsilon=0.0100
Episode 16000/20000 | X wins: 429 | O wins: 110 | Draws: 461 | epsilon=0.0100
Episode 17000/20000 | X wins: 462 | O wins: 126 | Draws: 412 | epsilon=0.0100
Episode 18000/20000 | X wins: 518 | O wins: 109 | Draws: 373 | epsilon=0.0100
Episode 19000/20000 | X wins: 530 | O wins: 115 | Draws: 355 | epsilon=0.0100
Episode 20000/20000 | X wins: 478 | O wins: 101 | Draws: 421 | epsilon=0.0100
```

Final Performance

- Plays near-optimally
- Rarely loses after sufficient training
- Often results in draws (expected optimal behavior)

Learned weights:

```
[ 70.0023 116.3298 -153.4779 90.6402 -11.5883 -2.7347 64.9274  
-118.8893 83.3819 -72.2744 91.2738 26.3834 67.8185 -41.1898  
92.9521 -43.5603 2.448 5.2971 21.9983 -111.3439 64.8904  
-2.849 133.3422 0. 0. 0. ]
```

Evaluation vs random opponent:

As X -> {'win': 1978, 'lose': 0, 'draw': 22}

As O -> {'win': 1603, 'lose': 36, 'draw': 361}

X		O
O	O	
X	X	X
X wins! Press R to restart.		
R: restart AI vs AI mode		

5. Comparison of Snake vs Tic-tac-toe

Aspect	Snake	Tic-tac-toe
Environment	Single-agent	Two-player
State space	Very large	Small
Difficulty	High	Moderate
Learning stability	Harder	Easier
Feature importance	Critical	Important
Final performance	Moderate	Near-optimal

6. Challenges Faced

Snake

- Looping behavior
- Local optima (safe but useless moves)
- Need for spatial awareness

Tic-tac-toe

- Handling opponent correctly
- Learning balanced strategy

7. Key Insights

1. Feature design is more important than the model itself
2. Linear models can work well with good features
3. Exploration is necessary for learning
4. Snake requires global awareness → harder problem
5. Tic-tac-toe is simpler and converges faster

8. Conclusion

In this assignment, we successfully implemented two learning agents using linear value function approximation.

- Snake demonstrated the challenges of large state spaces and the importance of advanced features.
- Tic-tac-toe showed that even simple linear models can achieve near-optimal play in smaller domains.

Overall, this assignment highlights the fundamental machine learning idea:

Generalization through feature-based function approximation instead of memorization.